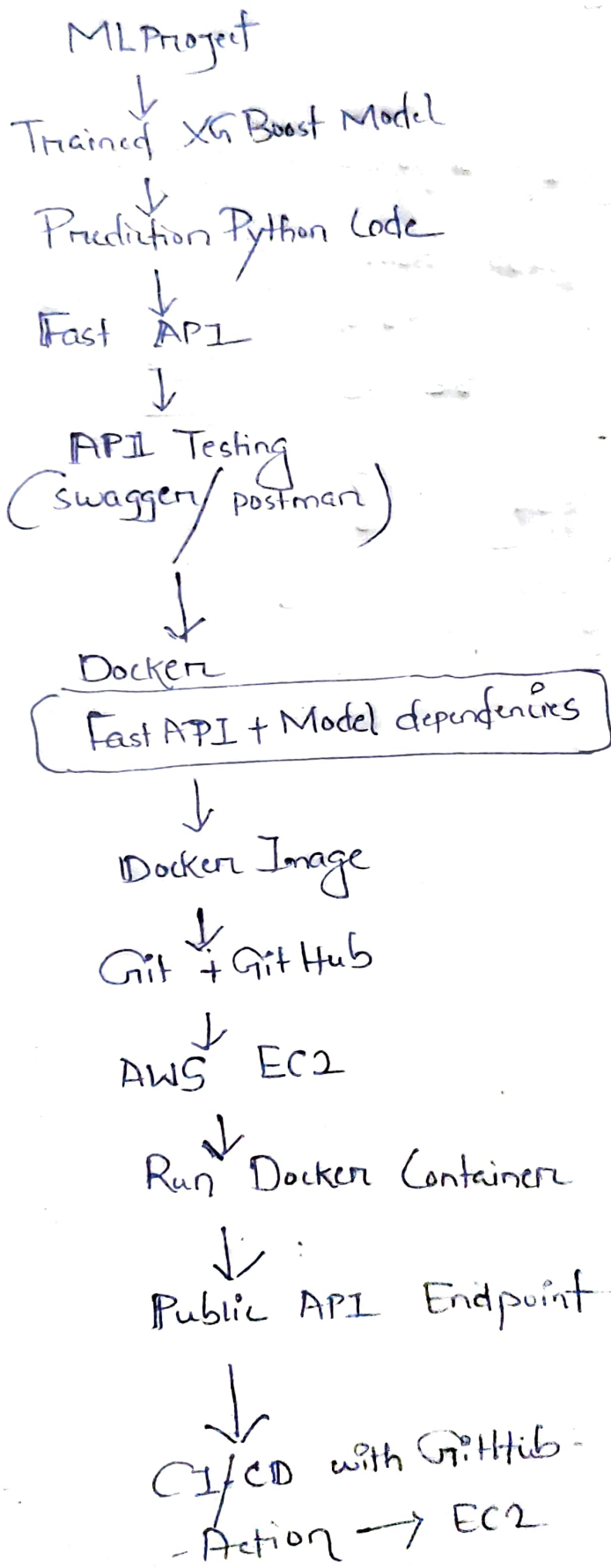


22/09/2026

Production Style ML Project deployment



Employee data

↓
Data Preprocessing

↓
XGBoost Model

↓
final-xgboost-employee-performance-model.joblib

→ This model is usable from Python/Notebook code.
Our goal is to turn it into a web accessible
(web accessible ML service)

1. What is ML deployment?

Suppose someone gives these employee details
in your application.

Age = 30
Department = Development
Job Role = Developer
Environment Satisfaction = 4
Salary hike = 18%
...
...

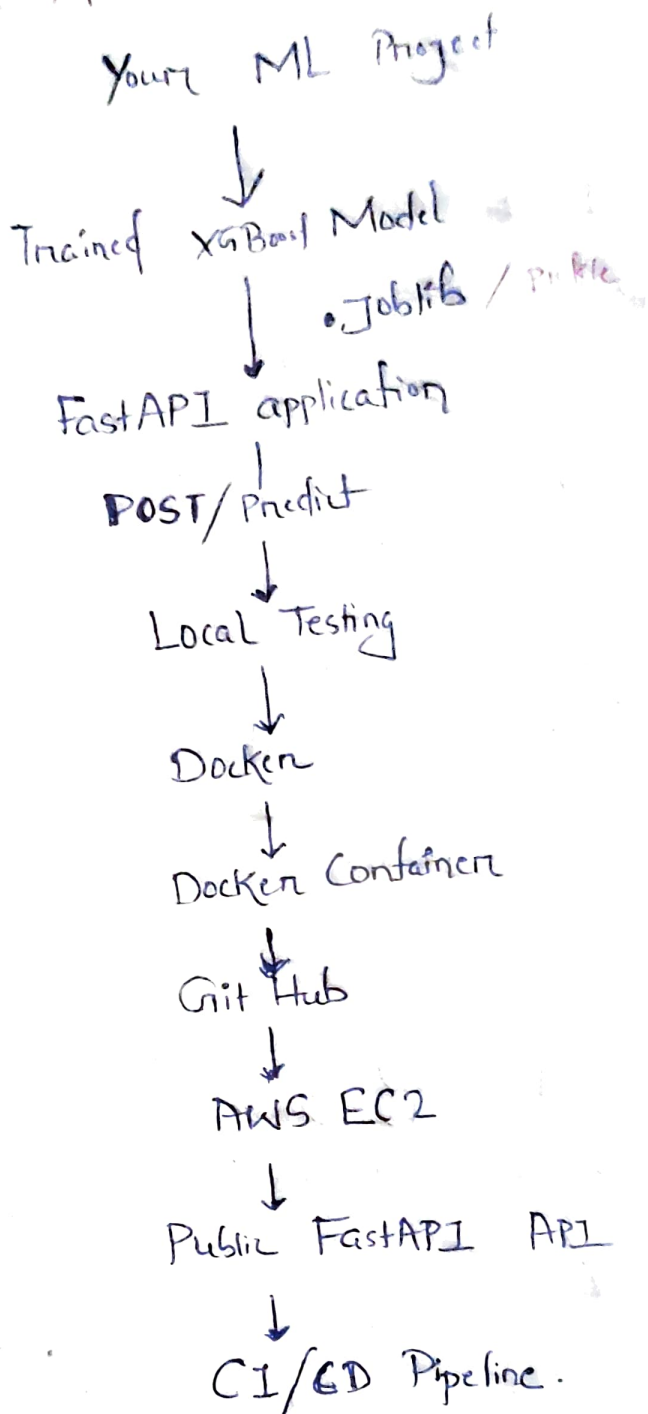
They shouldn't need your Jupyter Notebook.
Instead, they should be able to send
the information to an API: (POST/Predict)
and receive:

JSON

```
{  "Predicted performance rating": 3,  
    "Probabilities": {  "2": 0.0021,  
                        "3": 0.9855,  
                        "4": 0.0121
```

That's Model Deployment.
making the model available to an Application/user through a Service.

2) Complete Architecture :

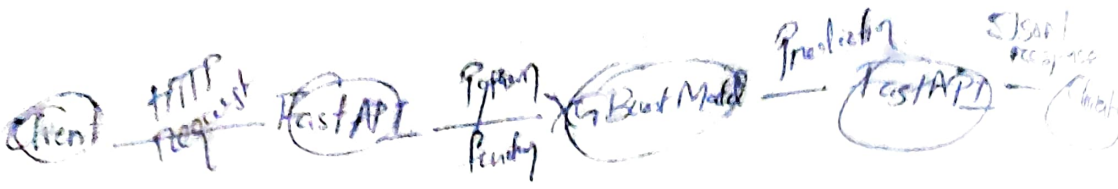


3) What is FastAPI ?

Your Model & your API are two things

XGBoost
26 features → Predict Performance

FastAPI Provides a way for another application to communicate with your model over HTTP.



4. What is an **API** ?

Application Programming Interface

we will create Endpoints

GET / health
POST / Predict

→ to check whether the application is running

Used to make Prediction
 - Client Send Employee Information
 - API sends it to model
 - model returns the Prediction

Return JSON

```

{
  "status": "healthy"
}
  
```

5. Why POST Instead of GET for Prediction ?

We have many Employee features : (Age, gender, Department)
 we need to send to server.
 for that we need POST request.

POST / Predict

Request Body — Employee Information — FastAPI — Model — Prediction

→ A GET request is generally used for retrieving information, whereas POST is commonly used when sending a request body for processing.

6) Where does Docker come in?

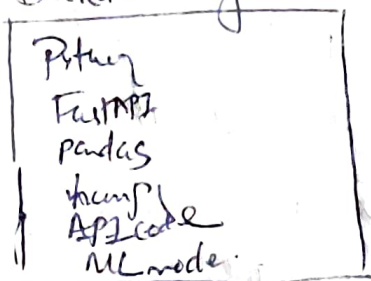
without docker, Imagine deploying your app to another Computer.

You might need:

- Python
- FastAPI
- pandas
- numpy
- Scikit Learn
- XGBoost
- Jobs
- ...

So that it will work on any machine. with Compatible Versions

Docker Image



↓
Container
↓
Running API

There is difference between Image & Containers

7) Where does AWS EC2 come in?

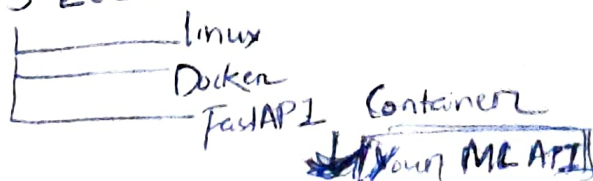
- Your Computer is not suitable for Permanent Public Server.

- AWS EC2 gives us a Virtual Server in Cloud.

Your Computer

↓ GitHub
GitHub Repo

↓
AWS EC2



Someone can communicate with the API over the Internet

8.) Where does Git/GitHub come in?

Git tracks changes to your code.

GitHub stores the repository remotely.

Local Computer

↓ git Push

Git Hub

↓

AWSEC2

Git Hub becomes Particularly Important when we Introduce CI-CD.

9.) What is CI/CD ?

Imagine You modify
api/main.py

Without Automation, you might have to:

change code

↓

git Push

↓

SSH into EC2

↓

git Pull

↓

docker build

↓

stop old docker

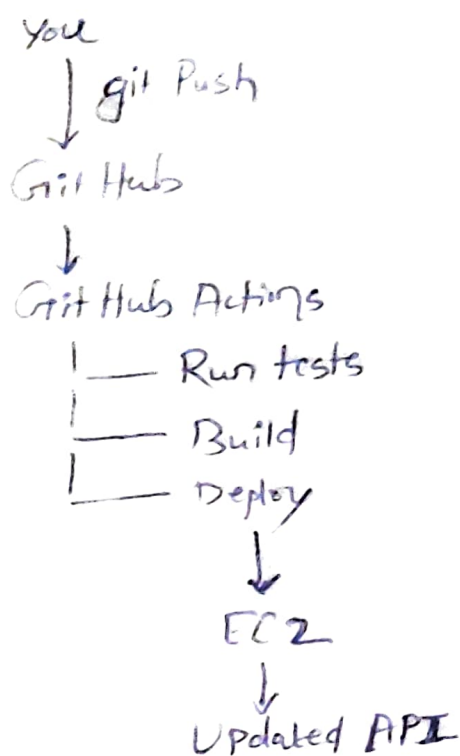
↓

start new container

This is Manual deployment.

So CI/CD:

With CI / CD :



So CI/CD automates the Software delivery Process.
we will build this Only after manual deployment works.

10. The Learning Sequence :

Step 1 - Understand Architecture

Model — API — Docker — GitHub — EC2 — CI/CD

Step 2 -

Create api/

├── main.py
└── schemas.py

Load your existing `joblib` model & expose `/predict`

Step 3 : Local API testing

we'll test

- /docs
- /health
- /predict

using Swagger UI and Possibly curl/Postman

Step 4 - Docker

Create:

Dockerfile
requirements.txt
• dockerrun

Then:

Dockerfile



docker build



image



docker run



Container

Step 5 - Git/GitHub

Create a Clean Git repository and
Push the deployment code.

Step 6 (- AWS EC2)

Create an EC2 instance, connect through
SSH, Install Docker, Pull the Project,
and Run the Container.

Step 7: (Public API)

We will expose the FastAPI application
through the EC2 instance.

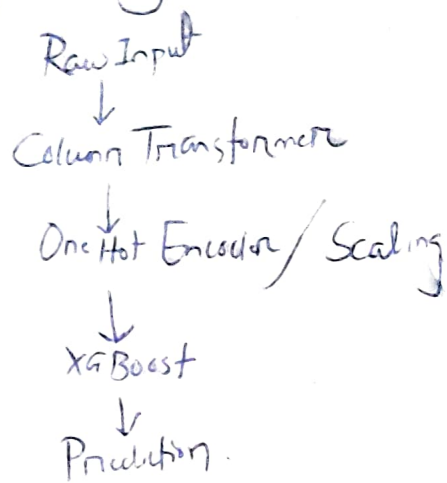
Steps: CI/CD

Finally:

git Push
↓
Git Hub Actions
test
↓
build
↓
deploy → EC2

One Important thing for my Project:

Our current trained pipeline already contains preprocessing:



That's why we saved the complete Pipeline, rather than only the XGBoost Classifier.

So our API does not need to manually ~~create~~ recreate all the preprocessing logic.

it can essentially do:

$$\text{Prediction} = \text{model.predict}(\text{input_data})$$

The saved Pipeline Handles the Preprocessing.

This is a good Production design for this particular project.

FastAPI - exposes our ML model through HTTP

API Endpoint - entry point such as `/Predict`

POST - Sends employee data for Prediction

Docker - Packages application + runtime/dependencies

Container - running instance of a Docker Image

Git - Version Control

GitHub - remote repository

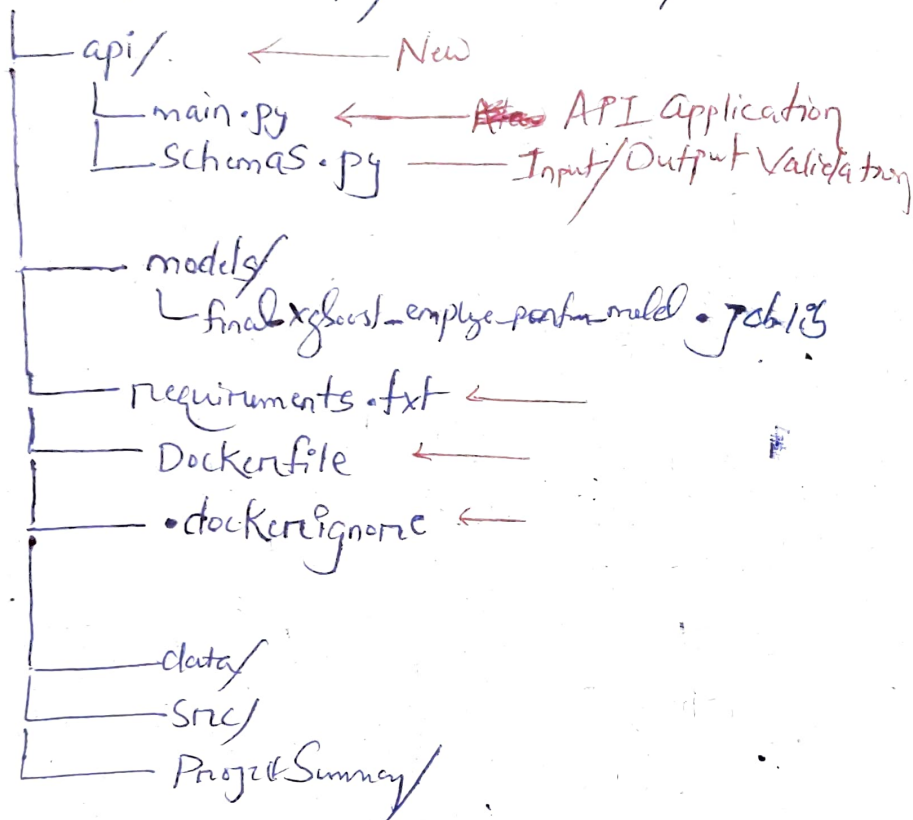
EC2 - cloud Virtual Server

CI/CD - automates testing/building/deployment

- Your existing project has the trained model and prediction notebook.
- We are adding Separate application that can receive employee information and return Prediction.

New deployment Structure

INX - Future - Employee - Performance /



We are not changing ~~the~~ your completed notebooks.
The API will load the Saved Pipeline from models

2.2) What is a FastAPI application?

Fast API is a Python framework for creating web APIs. Our application will eventually have two endpoints.

<u>End Point</u>	<u>Purpose</u>
GET/health	Check whether the Service is running
POST/ Predict	Receive employee data and return a prediction.

We'll use Pydantic Schemas to validate incoming data, so the API can reject missing fields or incorrectly formatted values before calling the Model.

main.py will contain the FastAPI app, Endpoints, model loading and prediction logic.

Schemas.py will define the employee input fields and validate incoming requests.

- Load the saved model when the application starts
- Provide a /health endpoint to check that it's running
- Accept Employee details at /predict
- Return the predicted rating and probabilities as JSON

27/09/2026

Docker: Package your ML API

Now we'll package the API and its dependencies so it can run in a consistent environment on another machine, including AWS EC2.

31 Understanding Docker Image vs. Container:

Docker Image:

A packaged template containing Python, dependencies, API code and Model.

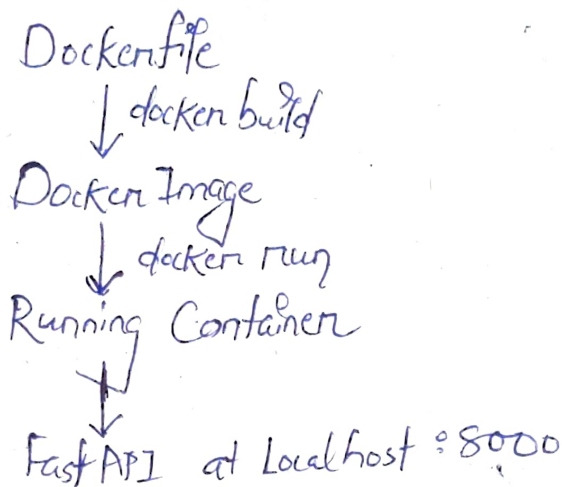
↓
docker run

Docker Container:

A running instance of that Image, serving your API

Think of an Image as a recipe/package and a Container as the running application made from it. You can create multiple containers from the same image.

Our Workflow :



- You have packaged your trained ML Model and FastAPI application into a Docker image & then started it as a container. The API is now accessible through a mapped port without needing to launch uvicorn directly from your Virtual environment.
- Now FastAPI + XGBoost Model is now running successfully inside Docker.
- Upload to gitHub
- Prepare AWS EC2 deployment.

- 1) Launch a free-tier-eligible EC2 instance if your account and region offer one.
- 2) Configure network access securely
- 3) Install & Run docker on the Instance
- 4) Transfer your application image or build it on EC2.
- 5) Test the Public API endpoint.

Step 6: Deploy your ML API on AWS EC2

Now we will move your Dockerized employee Performance Prediction API from your local Computer to an AWS EC2 Virtual Machine.

Deployment Workflow:

1. Launch an EC2 Instance
2. Configure secure network access.
3. Connect to the Instance
4. Install Docker
5. Run your API container
6. Test the public API